TERRA
DESENVOLVIMENTO AGROPECUÁRIO

PROJETO TERRA · PLATAFORMA UNIFICADA

Infraestrutura de produção

Como colocar o painel no ar para cerca de 50 acessos simultâneos sem dor de cabeça: a arquitetura, o tamanho dos servidores, as etapas da migração e a rotina de operação.

21 de outubro de 2026 · feito para leitura no celular

1. Resumo em um minuto

- **50 acessos simultâneos é carga pequena para este sistema.** Medimos: as páginas respondem em 20 a 50 ms e o boletim analítico em cerca de 0,4 s. O PDF é montado no navegador de quem exporta, não no servidor.
- **Sua intuição está meio certa.** O banco deve sair de dentro do Docker, mas pela segurança dos dados (backup, restauração, atualização), não por desempenho.
- **Backend e frontend continuam em Docker.** É justamente o que dá menos trabalho: a mesma imagem testada vai para produção, e atualizar ou voltar atrás é um comando.
- **Recomendação:** uma máquina virtual (VM) para a aplicação, com o Docker Compose que o projeto já tem, e um **PostgreSQL gerenciado** pelo provedor, os dois na região de São Paulo. Backups em dois lugares e um alerta externo se o site cair.
- **Prazo:** de 1 a 2 semanas de trabalho até a virada, com os BIs atuais funcionando em paralelo por 1 ou 2 meses para conferência.

UM PONTO A CONFIRMAR ANTES DE ESCOLHER O PROVEDOR

As migrações do banco usam um superusuário e o papel `app_owner`, que ignora o RLS (a regra que separa as fazendas por usuário). Bancos gerenciados não dão superusuário de verdade. Isso tem de ser testado no provedor escolhido (fase 1 da migração). Se não funcionar, a opção C (VM própria só para o banco) resolve sem mudar nada no sistema.

2. O que medimos

Medições feitas em 21/10/2026 na API de desenvolvimento (um único processo, sem otimização), com o banco de validação completo.

O QUÊ	RESULTADO
Abrir o menu (lista de fazendas do usuário)	**83 ms**
Páginas (movimentação, vendas, financeiro, previsto × realizado)	**22–46 ms**
Página de um grupo de 6 fazendas	**41 ms**
Boletim analítico (dados do PDF)	**≈ 0,4 s**
20 pedidos de página ao mesmo tempo	**0,6 s, 0 erros**
10 boletins analíticos ao mesmo tempo	**2,9 s, 0 erros**
Tamanho do banco (230 planilhas e 2 clientes Metryx)	**437 MB**

O que isso quer dizer para 50 pessoas

Quem está lendo uma tela não gera pedido. Cada clique gera de 1 a 3 pedidos. Com 50 pessoas navegando ao mesmo tempo, o servidor recebe na ordem de **2 a 7 pedidos por segundo**, e um único processo de desenvolvimento já atendeu 36 por segundo.

O caso mais pesado é muita gente exportando o boletim analítico no mesmo minuto. Mesmo assim, cada exportação custa menos de meio segundo de servidor, porque a imagem do PDF é feita no navegador.

A máquina de desenvolvimento é mais forte que a VM proposta. Por isso a recomendação usa 4 processos na API (hoje são 2) e folga de memória, e o teste de carga da fase 4 confirma na VM de verdade.

3. Docker sem mistério

O que é

No Linux, um container **não é uma máquina virtual**. É um programa comum do sistema, rodando isolado, com tudo de que precisa (Python, bibliotecas) dentro da imagem. Por isso o custo de desempenho é desprezível: para a máquina, a API em Docker é praticamente o mesmo que a API instalada direto.

O que ele resolve

- A imagem testada aqui é a mesma que roda em produção. Não existe o "na minha máquina funciona".
- Atualizar é `docker compose up -d`. Voltar à versão anterior é subir a imagem anterior.

- Cada serviço tem as dependências dele, sem brigar com o sistema operacional.
- O `docker-compose.prod.yml` do projeto já existe e foi validado (guia 03).

Onde ele exige cuidado

Nos **dados guardados dentro dele**. Com o banco num container, a atualização de versão do Postgres, o backup, o disco e a restauração ficam inteiramente por sua conta, e banco e aplicação caem juntos se a VM tiver problema.

> **E RODAR BACKEND E FRONTEND DIRETO NA VM, SEM DOCKER?**
>
> Não recomendo. Seria preciso instalar Python 3.12, Node e Caddy, configurar quatro serviços no systemd (API, worker, beat e o servidor web) e repetir tudo a cada servidor novo ou atualização. É mais trabalho, sem ganho de desempenho. O frontend nem é um programa rodando: é um conjunto de arquivos que o Caddy entrega.

> **CONCLUSÃO**
>
> Aplicação em Docker, banco fora dele. É exatamente a divisão que você intuiu, só que sem tirar o Docker da aplicação.

4. A arquitetura recomendada

Usuários
navegador, no computador ou no celular

▼ HTTPS (porta 443)

VM de aplicação · Docker Compose
Ubuntu 24.04 LTS, região São Paulo

- **web**: Caddy, o painel e o HTTPS automático
- **api**: FastAPI, 4 processos
- **worker**: as cargas, uma por vez
- **beat**: dispara as cargas no horário
- **redis**: só a fila das cargas
- **planilhas**: rclone, traz do Drive

▼ rede privada do provedor, com SSL

PostgreSQL 16 gerenciado
backup automático com restauração em qualquer ponto dos últimos dias; atualizações de segurança feitas pelo provedor; sem acesso pela internet

▼ cópia diária

Armazenamento de backups
um bucket (S3 ou equivalente), fora da VM e do banco, com 30 dias de retenção

Por que assim

- **Os dados ficam no lugar mais seguro.** Se a VM quebrar, o banco continua intacto; uma VM nova sobe em menos de uma hora com o compose e o arquivo de configuração.

- **Nada novo para aprender.** A aplicação é o compose que já existe, só sem o serviço do banco.
- **O Redis não guarda nada importante.** É só a fila; o resultado das cargas fica no banco. Pode continuar na VM.
- **Só o Caddy fica exposto** (portas 80 e 443). A API, o Redis e o banco não aparecem na internet.

5. As três opções

A · Tudo numa VM, como está hoje

Como	O `docker-compose.prod.yml` atual, com o banco num container.
Prós	Já está pronto; o mais barato; uma máquina só.
Contras	Banco e aplicação caem juntos. Backup, restauração e atualização do Postgres são por sua conta. O backup diário fica na própria VM até alguém copiar para fora.
Quando	Para começar rápido ou para um piloto, desde que os backups sejam copiados para fora todo dia.

B · VM com Docker + PostgreSQL gerenciado

RECOMENDADA

Como	A aplicação no compose, numa VM; o banco no serviço gerenciado do mesmo provedor (RDS na AWS, Cloud SQL no Google, Azure Database etc.).
Prós	Backup automático com restauração em qualquer ponto no tempo, atualizações e monitoramento do banco por conta do provedor. É a opção "sem dor de cabeça".
Contras	Custa mais que uma VM do mesmo tamanho. Não tem superusuário de verdade: é preciso confirmar que as migrações e o papel `app_owner` funcionam (fase 1).
Quando	Quando ninguém da equipe quer ser responsável por administrar banco de dados.

C · VM com Docker + VM só para o banco

Como	Duas VMs: a da aplicação e outra com o PostgreSQL instalado direto (sem Docker), com o pgBackRest cuidando dos backups.
Prós	Controle total (superusuário, configuração, extensões); funciona sem mudar nada no sistema; costuma sair mais barato que o gerenciado.
Contras	Backup, restauração, atualização e monitoramento do banco ficam com vocês, como na opção A, só que numa máquina separada.
Quando	Se o provedor gerenciado não aceitar as migrações, ou se houver alguém com prática em PostgreSQL.

E KUBERNETES, VÁRIOS SERVIDORES, BALANCEADOR?

Não é necessário para 50 usuários. Seria mais caro e mais complicado, sem benefício no tamanho atual. A seção 12 diz quando pensar nisso.

6. Tamanho dos servidores

ITEM	RECOMENDAÇÃO INICIAL
VM de aplicação	2 vCPU, 4 GB de RAM, 50 GB de SSD, Ubuntu 24.04 LTS. Subir para 4 vCPU e 8 GB se as cargas de planilhas deixarem o painel lento (aumentar uma VM leva minutos).
PostgreSQL	Versão 16, 2 vCPU, 4 GB de RAM, 20 GB de disco com crescimento automático. Backup automático de 7 a 14 dias, com restauração em ponto no tempo.
Backups	Um bucket com 30 dias de retenção (poucos GB).
Domínio	Um subdomínio, por exemplo `painel.terra.agr.br`, apontando para a VM.

Por que esse tamanho basta

O banco inteiro tem 437 MB e cresce devagar: uma safra a mais de planilhas e alguns clientes Metryx somam pouco. A memória da VM é dominada pelo worker durante as cargas, quando ele lê as planilhas.

Onde contratar

Provedores com região em São Paulo (latência baixa para Cuiabá e dados no Brasil, o que simplifica a LGPD): AWS, Google Cloud, Microsoft Azure, Oracle Cloud e Magalu Cloud. Para a opção B, AWS (VM + RDS) e Google Cloud (VM + Cloud SQL) são os mais documentados.

> **SOBRE PREÇO**
>
> Não cito valores porque mudam com o tempo e entre provedores. Simule nas calculadoras de preço de cada um com o tamanho acima. Como referência, o banco gerenciado costuma custar mais que uma VM equivalente; a diferença paga o backup, a restauração e a atualização feitos por eles.

7. O que muda no projeto

Pouca coisa, e tudo pequeno. Para a opção B:

1. **Tirar o serviço** `db` do compose de produção e apontar `POSTGRES_HOST` para o endereço privado do banco gerenciado.
2. **Conexão com SSL.** Hoje a URL do banco não tem essa opção (`backend/app/core/config.py`); é um parâmetro a acrescentar (`sslmode=require`).
3. **Criar os papéis uma vez.** O script `docker/postgres/init/01-papeis.sh` só roda quando o volume do Docker nasce. No gerenciado, o mesmo SQL é executado uma vez pelo usuário administrador. É aqui que se confirma o `BYPASSRLS` do `app_owner`.
4. **API com 4 processos** (hoje 2, no `backend/Dockerfile`).
5. **Limite no tamanho dos logs** dos containers. Hoje não há limite, e os logs podem encher o disco com o tempo.
6. **Backup para fora.** O dump diário passa a ser copiado para o bucket. Hoje ele fica só na VM.

7. **Alerta externo** consultando `/api/v1/health` a cada minuto (UptimeRobot, Better Stack ou o monitor do provedor).
8. **Depois (recomendado):** GitHub Actions rodando os testes e gerando as imagens, para a VM só baixar e subir, sem compilar nada lá.

Na opção C, os itens 1 e 2 mudam pouco (o host vira o IP privado da VM do banco) e o item 3 é o próprio script, rodado como superusuário.

8. Etapas da migração

Fase 0 · Decisões

1 dia

- Provedor, região e a opção (B ou C);
- o domínio do painel;
- como as planilhas chegam ao servidor (decisão D8; a recomendação é o rclone com uma conta de serviço do Google, só leitura);
- quem recebe os alertas e a data da virada.

Pronto quando: as quatro decisões estão escritas.

Fase 1 · Montar a infraestrutura

1 a 2 dias

- Criar o banco (rede privada, backup automático, alerta de disco);
- criar a VM, o firewall (22, 80 e 443), o Docker e o DNS;
- criar o bucket de backups;
- **testar os papéis e uma migração no banco gerenciado**, o ponto de atenção do resumo.

Pronto quando: `alembic upgrade head` roda limpo no banco novo.

Fase 2 · Ajustar o projeto

1 a 2 dias

Os itens da seção 7, com os testes passando e o guia de deploy (`docs/03_DEPLOY_VPS.md`) atualizado.

Pronto quando: o compose sobe na VM apontando para o banco gerenciado.

Fase 3 · Levar os dados

1 dia

Ver a seção 9. Em resumo: restaurar o banco de validação, limpar o que é de teste e cadastrar de novo os tokens do Metryx com a chave de produção.

Pronto quando: os números do painel batem com os do banco de validação.

Fase 4 · Ensaio geral

1 a 2 dias

- Conferir os números com os scripts de validação (`backend/validacao/cruzar_*`);
- teste de carga com 50 usuários virtuais navegando e exportando (k6 ou Locust);
- restaurar um backup num banco novo e conferir;
- reiniciar a VM e ver tudo voltar sozinho.

Pronto quando: os quatro testes passaram e estão anotados.

Fase 5 · Virada

1 dia

- Última carga das planilhas;
- criar o admin real e importar a equipe (`importar-equipe`);
- avisar os usuários e mandar o endereço.

Pronto quando: a equipe entrou e viu as suas fazendas.

Fase 6 · Convivência com os BIs

1 a 2 meses

Os dois Power BIs continuam no ar. A equipe usa o painel novo e confere com os BIs o que estranhar. Pendências e erros de carga são acompanhados toda semana. Depois disso, os BIs são desligados.

Pronto quando: um fechamento de mês inteiro foi feito só pelo painel novo.

9. Como levar os dados

Caminho recomendado: restaurar o banco validado

O banco de validação já tem tudo conferido contra os BIs e o Metryx: as 230 planilhas, o histórico do snapshot do BI (que só existe na máquina Windows), as cargas reais do Metryx e os de-paras classificados.

1. Gerar um dump do `terra_validacao` (`pg_dump --format=custom`);
2. restaurar no banco de produção, depois de criar os papéis;
3. **apagar os usuários de teste** (o admin e o consultor de desenvolvimento);
4. **apagar as credenciais do Metryx** e cadastrar de novo com a chave de cifra de produção, que deve ser nova e diferente da de desenvolvimento (com a autorização de cada cliente);
5. rodar a carga de planilhas uma vez, para pegar o que mudou desde a validação.

Alternativa: carregar do zero

Subir o banco vazio e deixar as cargas trazerem tudo: planilhas pelo worker; o histórico (Q6) rodado da máquina Windows por um túnel SSH, como já está no guia 03; o Metryx com os tokens. Fica mais limpo, mas é mais demorado e os de-paras precisam ser classificados de novo.

NOS DOIS CASOS

Os segredos de produção (senhas do banco, a chave do login e a chave de cifra do Metryx) são novos e ficam num cofre de senhas (1Password, Bitwarden), não só no arquivo da VM.

10. Rotina de operação

QUANDO	O QUÊ
Sozinho, todo dia	Backup do provedor e dump copiado para o bucket; cargas de planilhas a cada 2 horas; alerta se o site cair; patches de segurança do Ubuntu (`unattended-upgrades`).
Toda semana	Olhar a aba Pendências e as cargas com erro; ler os alertas da semana.
Todo mês	Restaurar um backup num banco de teste (backup não testado não é backup); conferir espaço em disco; atualizar as imagens base.
A cada versão	Atualizar o código e rodar `up -d` : a migração roda sozinha. Antes, garantir um backup recente; voltar atrás é subir a versão anterior.

Se algo der muito errado

PROBLEMA	RECUPERAÇÃO
A VM quebrou ou foi perdida	VM nova, Docker, o código e o arquivo de configuração do cofre: em menos de 1 hora. Nenhum dado se perde, porque o banco está fora.
Dados apagados ou estragados	Restaurar o banco no ponto no tempo anterior ao problema (gerenciado) ou o dump da véspera (bucket).
Uma atualização quebrou o painel	Subir a imagem anterior. Se a migração mexeu em dados, restaurar o backup feito antes da atualização.

11. Segurança

- Firewall aberto só nas portas 80 e 443 para todos, e 22 (SSH) só com chave, de preferência só dos IPs da equipe;
- o banco sem endereço público: só a VM fala com ele, pela rede privada e com SSL;
- o RLS continua de verdade: a API conecta como `app_user` e as cargas como `app_etl`, sem poder ignorar as regras;
- os segredos fora do git (como já é) e guardados num cofre de senhas;
- os backups no bucket com acesso restrito e criptografados;
- o rclone com permissão só de leitura no Google Drive;

- a documentação da API (`/docs`) continua fora da internet (o Caddy já não a publica);
- os tokens do Metryx continuam cifrados no banco e só com a autorização de cada cliente.

12. Pendências e futuro

O que não é infraestrutura, mas trava a virada

- **D8:** como as planilhas chegam ao servidor (Google Drive pelo rclone com conta de serviço);
- **Metryx:** autorização de cada cliente para ligar a carga diária;
- **Equipe:** a lista de técnicos e as fazendas de cada um (a planilha de levantamento);
- **D10:** permissões por função ficaram para depois; hoje quem não é admin vê as fazendas que lhe foram dadas.

Quando pensar em crescer

- Painel lento em horário de pico, ou mais de 200 pessoas ao mesmo tempo: primeiro aumentar a VM (minutos); depois, guardar em cache o boletim analítico por alguns minutos;
- cargas atrapalhando quem usa o painel: separar o worker numa segunda VM pequena;
- banco no limite: aumentar o plano do gerenciado ou criar uma réplica só de leitura;
- vários servidores com balanceador, ou Kubernetes, só muito além disso.

13. Glossário rápido

VM
Máquina virtual: um servidor alugado no provedor.

Imagem / container
A imagem é o pacote com o programa e tudo de que ele precisa; o container é essa imagem rodando.

Docker Compose
O arquivo que diz quais containers sobem e como se ligam (`docker-compose.prod.yml`).

Banco gerenciado
PostgreSQL em que o provedor cuida da máquina, do backup e das atualizações; você só usa.

Restauração em ponto no tempo
Voltar o banco a como estava num minuto exato do passado (por exemplo, logo antes de um erro).

Bucket
Uma pasta na nuvem para guardar arquivos (os backups).

RLS
A regra do banco que só deixa cada usuário ver as fazendas dele.

Caddy
O servidor web que entrega o painel e renova o certificado HTTPS sozinho.